

From Perceptron to MLP — Handwritten Digit Recognition**Due date: 09/22/2026 @ 11:59 PM****Project Overview**

In class, we developed a Single-Layer Perceptron (SLP) and applied it to the [scikit-learn](#) handwritten digits dataset. We examined how a perceptron calculates its output, learns from errors, and updates its weights and biases.

In this project, you will extend what we developed in class to the larger MNIST handwritten digit dataset. You will first adapt the SLP implementation from class to classify MNIST digits and investigate whether and how the learning rate affects its learning process and performance. You will then develop a Multilayer Perceptron (MLP), experiment with different network architectures, and compare the performance and generalization of the SLP and MLP models.

The purpose of this project is to investigate how learning parameters and network architecture affect model performance and generalization, with particular emphasis on how adding hidden layers changes the ability of a neural network to learn complex patterns.

Learning Objectives

By completing this project, you should be able to:

- *Apply a machine learning model to a new dataset.*
- *Adapt a Single-Layer Perceptron to a different input representation.*
- *Investigate whether and how the learning rate affects SLP training and performance.*
- *Build and train an MLP using scikit-learn.*
- *Experiment with different neural network architectures.*
- *Evaluate classification models using appropriate performance measures.*
- *Analyze correct and incorrect predictions.*
- *Compare the capabilities and limitations of SLP and MLP models.*
- *Interpret experimental results and draw evidence-based conclusions.*

1. Dataset: MNIST

You will use the MNIST handwritten digit dataset, which contains images of handwritten digits from 0 through 9.

Each MNIST image is:

$$28 \times 28 = 784 \text{ pixels}$$

Therefore, each image can be represented using 784 input features.

Department of Math, Computer Sci, & Data Sci

The classification problem has 10 possible output classes:

0,1,2,...,9

You may load MNIST dataset using:

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml(
    "mnist_784",
    version=1,
    as_frame=False
)

X = mnist.data
y = mnist.target.astype(int)
```

Data Preparation

Examine the dataset before training your models. At minimum, determine:

- Number of examples
- Number of features
- Possible target values
- Range of pixel values

Scale/normalize the input features (pixel values) to the range 0–1 using $X = X / 255$.

Then create training and testing sets. Use the same train/test data for both SLP and MLP so that the comparison is fair. Use the same [train_test_split](#) we had in class, 80% for training and 20% for testing.

Part I: Single-Layer Perceptron

2. Adapt the SLP Developed in Class

You are not required to implement the Single-Layer Perceptron from scratch.

Start with the SLP implementation developed together in class and adapt it to work with MNIST dataset.

Our classroom dataset contained 8 x 8 images → 64 input features

MNIST dataset contains 28 x 28 images → 784 input features

Therefore, your SLP architecture becomes: → 784 by 10

You will need to identify and make the necessary modifications to the classroom implementation.

Your adapted SLP should continue to follow the step-by-step learning process discussed in class.

Department of Math, Computer Sci, & Data Sci

SLP Experiments

Investigate whether and how the learning rate affects the performance of the SLP.

Train the SLP model three times using the following learning rates:

0.001, 0.01, and 1.0

Keep all other settings the same, including the train/test split, random seed, and number of epochs, so that the comparison is meaningful. Only the learning rate should change between the three experiments.

For each learning rate, observe the model's training behavior and final performance. Changing the learning rate may or may not result in different classification accuracy. Your goal is to observe, report, and analyze the results rather than obtain a particular outcome.

SLP Results

For each learning rate, report:

- Learning rate
- Number of epochs
- Training accuracy
- Test accuracy

Summarize the main results in a table similar to the following:

Learning Rate	Epochs	Training Accuracy	Test Accuracy
0.001			
0.01			
1.0			

Also, show how the number of patterns with errors changes across epochs for each learning rate. You may present this using a table or plot.

Briefly discuss your observations. Address the following questions:

- Did changing the learning rate affect the training process?
- Did it affect the final training or test accuracy?
- Which learning rate produced the best results, if any?
- If the results were similar or unchanged, what did you observe about the learning process?

Department of Math, Computer Sci, & Data Sci

Part II: Multilayer Perceptron**3. Build an MLP**

Unlike the SLP, the MLP contains at least one **hidden layer**.

Next, develop a Multilayer Perceptron using:

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(64,),
    max_iter=100,
    random_state=42
)
```

Train the model using the same training data used for the SLP and evaluate it using the same test data.

4. Experiment with MLP Architectures

Machine learning involves making and evaluating design decisions. You will investigate how changing the hidden-layer architecture affects model performance.

Train and evaluate the following MLP architectures:

 $784 \rightarrow 32 \rightarrow 10$ $784 \rightarrow 64 \rightarrow 10$ $784 \rightarrow 128 \rightarrow 10$

You may also investigate a network with more than one hidden layer, such as:

 $784 \rightarrow 64 \rightarrow 32 \rightarrow 10$

You are encouraged to select architectures that allow you to make a meaningful comparison rather than simply trying increasingly large networks.

Record your results in a table similar to the following:

Model	Hidden Layer(s)	Architecture	Training Accuracy	Test Accuracy
SLP	None	$784 \rightarrow 10$		
MLP 1	32	$784 \rightarrow 32 \rightarrow 10$		
MLP 2	64	$784 \rightarrow 64 \rightarrow 10$		
MLP 3	128	$784 \rightarrow 128 \rightarrow 10$		
MLP 4	64, 32	$784 \rightarrow 64 \rightarrow 32 \rightarrow 10$		

Identify your best-performing MLP based on its performance on unseen data.

Department of Math, Computer Sci, & Data Sci

Part III: Model Evaluation and Comparison**5. Evaluate Your Models**

Compare the SLP with your best MLP.

At minimum, report:

Training accuracy

Test accuracy

Confusion matrix

Use a confusion matrix to investigate the types of mistakes made by each model.

You may use:

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

Your analysis should address questions such as:

- Which digits are most frequently confused?
- Does the SLP struggle with particular digits?
- Does the MLP improve the classification of those digits?
- Is there a noticeable difference between training and test accuracy?

Do not simply present the numbers. Explain what the results tell you about the models.

6. Examine Individual Predictions

Visualize several examples from the test set.

Include at least:

- 3 correctly classified images
- 3 incorrectly classified images

For each image, clearly display:

- The handwritten digit
- Actual class
- Predicted class

For example:

```
plt.imshow(X_test[i].reshape(28, 28), cmap="gray_r")  
plt.title(f"Actual: {y_test[i]} Predicted: {y_pred[i]}")  
plt.axis("off")  
plt.show()
```

Discuss the incorrect predictions. Consider whether some misclassified digits visually resemble the predicted digit.

Department of Math, Computer Sci, & Data Sci

Part IV: Analysis**7. SLP vs. MLP**

Using your experimental results and what we discussed in class, compare the Single-Layer Perceptron and Multilayer Perceptron.

Your discussion should address:

- Network Architecture: How is the architecture of the MLP different from the SLP?
- Hidden Layers: What role do hidden layers play?
- Performance: Which model achieved better test performance? How large was the difference?
- Model Complexity: How did changing the number of hidden neurons and adding a second hidden layer affect model performance?
- Generalization: Compare training and testing performance. What does this tell you about how well the models generalize to unseen images?
- Limitations: What limitations did you observe in the SLP? How did the MLP address some of these limitations?

Your discussion should be based on your own experimental results, not simply definitions of SLP and MLP.

Project Report

Submit a concise technical report containing the following sections:

1. Introduction

Briefly describe the problem and the objective of the project.

2. Dataset and Data Preparation

Describe MNIST, its features and target, preprocessing, and the train/test split.

3. Single-Layer Perceptron

Describe how you adapted the classroom SLP to MNIST, its architecture, training settings, and results.

4. Multilayer Perceptron

Describe your MLP implementation and the architectures you investigated.

5. Experimental Results

Present your results clearly using appropriate tables and figures.

6. Model Evaluation

Include the confusion matrices and selected correct/incorrect predictions.

7. SLP vs. MLP Discussion

Compare the models and interpret the results.

8. Conclusion

Summarize your major findings and what you learned from the project.

Department of Math, Computer Sci, & Data Sci

Submission Requirements

Submit two separate files:

1. **Code:** One (.py) file or Jupyter Notebook (.ipynb) containing your complete implementation.
2. **Report:** One PDF containing your report, results, tables, figures, and analysis.

Your code must:

- Run without errors.
- Be clearly organized and appropriately commented.
- Clearly separate data preparation, SLP training/testing, MLP training/testing, and evaluation.
- Produce the results reported in your report.

Your report should focus on results and interpretation rather than reproducing large amounts of source code.

Individual Work Policy

This is an individual project.

The SLP implementation developed in class may be used as the starting point for Part I. You are expected to understand that implementation and independently adapt it to MNIST.

You are highly encouraged to discuss general machine learning concepts and course material with classmates and to help one another understand the concepts. However, all code modifications, experiments, results, figures, analysis, and written work submitted for this project must be your own.

Any use of AI tools must follow the course AI Assistance Policy. You are responsible for understanding, validating, and being able to explain all submitted code, analyses, and results.

Evaluation

Component	Weight
MNIST preparation and ML workflow	10%
SLP adaptation to MNIST	10%
SLP learning-rate experiment	15%
MLP implementation	10%
MLP architecture experiments	20%
Model evaluation and visualizations	15%
SLP vs. MLP analysis and interpretation	15%
Code/report quality and organization	5%
Total	100%